



Prompt Enhancement Accelerator

A complete guide to the architecture,
components, APIs, and data flow of the Prompt
Enhancement Accelerator built on TrueFoundry.



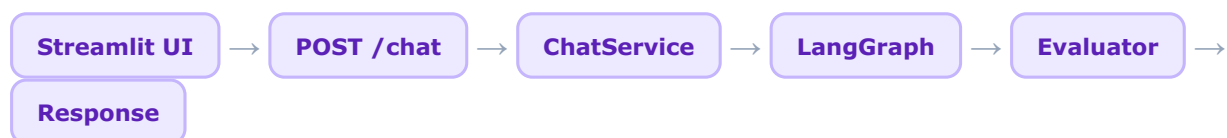
1. System Overview

Prompt Enhancement Accelerator is an end-to-end platform for evaluating, enhancing, and testing LLM system prompts. It combines a backend service with a Streamlit frontend and integrates deeply with TrueFoundry for prompt versioning and production trace analysis.

Top-Level Structure

```
prompt-validator/  
├─ src/           # Backend – FastAPI + LangGraph  
├─ ui/            # Frontend – Streamlit (5 tabs)  
├─ cli/           # CLI evaluation pipeline  
├─ trace/         # Production trace fetching & parsing  
├─ prompts/       # LLM system prompt files (.txt)  
├─ config/        # application.yaml, log-config.yaml  
└─ artifact/      # Static assets (logo)
```

Request Flow



8 Request Types

Type	Purpose
<code>validation</code>	Score prompt on 5 criteria OR apply recommendations → enhanced prompt
<code>verify_tests</code>	Run DeepEval metric evaluation on test cases
<code>verify_tests_exact</code>	Exact string match evaluation
<code>llm_judge</code>	Compare original vs enhanced prompt outputs side-by-side
<code>get_behavioral_recommendations</code>	Analyze production trace examples for improvement areas
<code>generate_suggestions</code>	Generate priority-ranked (HIGH/MEDIUM/LOW) improvement suggestions
<code>arena_comparison</code>	Head-to-head prompt comparison via DeepEval arena
<code>deepeval_prompt_metrics</code>	Custom GEval / PromptAlignment metrics



2. Backend (src/)

2.1 Controller — src/chat/controller/chat_controller.py

FastAPI Application Entry Point

Defines two endpoints, handles request logging, timing, and error handling.

Endpoint	Description
POST /chat	Main evaluation endpoint — accepts PromptRecommendationRequest, returns PromptRecommendationResponse
POST /traces/ fetch	Fetches live ChatCompletion spans from a TrueFoundry tenant, returns parsed TraceRecord list

2.2 Service — src/chat/service/chat_service.py

ChatService

Orchestrates the full evaluation workflow. Builds the LangGraph state machine, executes it, and extracts the final response.

- **get_chat_response(request)** — async entry point, runs the graph
- **_parse_with_sanitizer()** — JSON parsing with LLM output sanitization fallback
- **_get_last_ai_message()** — extracts final AI message from graph event stream

2.3 LangGraph — src/chat/graph/primary_graph.py

State Machine Orchestration

Routes requests to the correct handler based on `request.type`. All handlers share a common state schema.





2.4 Evaluators — src/chat/graph/

`llm_judge_evaluator.py`

LLM-as-Judge: compares original vs enhanced prompt outputs across 10 configurable metrics (0.0–1.0 scale). Overall = mathematical average.

`deepeval_evaluator.py`

DeepEval integration: GEval, Bias, Toxicity, PromptAlignment metrics. Runs test cases and returns pass/fail with scores.

`exact_match_evaluator.py`

Exact string match evaluation. Tests whether model response contains expected output string.

`arena_evaluator.py`

Head-to-head arena comparison. Uses natural language criteria to pick winner per test case.

2.5 LLM Judge — 10 Evaluation Metrics

Metric	Group	Description
<code>clarity</code>	General	How clear, well-structured, and easy to understand is the response?
<code>completeness</code>	General	How thoroughly does it address the user's request?
<code>accuracy</code>	General	How factually correct and relevant is the content?
<code>conciseness</code>	General	Appropriately concise without losing important detail?
<code>professional_tone</code>	General	How professional and contextually appropriate is the tone?
<code>output_format_compliance</code>	Guardrails	Does the response strictly follow the required output format or schema?
<code>hallucination_avoidance</code>	Guardrails	Did it avoid making up facts, policies, or services that do not exist?
<code>answer_relevance</code>	Conversational	Is the response directly relevant to what the user asked?
<code>prompt_instruction_adherence</code>	Conversational	Did the model follow all rules in the system prompt?



2.6 Data Models

PromptRecommendationRequest

- `session_id`, `type`
- `prompt_fqn` or `system_prompt`
- `model_name`, `temperature`, `max_tokens`
- `recommendations` — list of strings to apply
- `test_cases` — list of input/expected pairs
- `trace_examples` — production trace samples
- `reasoning_effort` — none/low/medium/high

PromptRecommendationResponse

- `session_id`, `status_code`
- `content.eval_result` — scores + recommendations
- `content.final_prompt_result` — enhanced prompt messages
- `content.test_evaluation_result` — judge/test results

2.7 Common Module — `src/common/`

Shared Infrastructure

- **`app_config.py`** — loads config/application.yaml, merges env vars, provides singleton
- **`prompt_service.py`** — TrueFoundry prompt integration: fetch FQN details, execute against LLM
- **`logger.py`** — centralized logging with file + console handlers
- **`checkpointer_factory_util.py`** — factory for in-memory / Redis / Postgres state persistence
- **`redis_pool_manager.py`** — Redis connection pooling for async checkpointer



3. Frontend (ui/)

A multi-tab Streamlit application. Global CSS from `style.css` is injected at app level. The Enhance tab uses `@st.fragment` for scoped reruns.

3.1 Application Entry — `ui/app.py`

- Renders 5-tab interface with TrueFoundry hero header
- Calls `init_session_state()` on startup (~90 keys)
- Loads and injects `style.css` globally via `st.markdown`

3.2 Tabs

Tab	File	Purpose
 Recommendations	<code>tabs/recommendations.py</code>	Fetch prompt by FQN or paste, score on 5 criteria, view recommendations
 Enhance	<code>tabs/enhance.py</code>	Apply recommendations → enhanced prompt; LLM-judge evaluation; suggestions; arena compare
 DeepEval Tests	<code>tabs/test_eval.py</code>	Upload test cases JSON, run DeepEval/exact-match, view pass/fail results
 Exact Match	<code>tabs/test_eval.py</code>	Exact string match evaluation with classwise metrics
 Trace Eval	<code>tabs/trace_eval.py</code>	Fetch live TFY traces, run LLM-judge on production examples, apply suggestions

3.3 Enhance Tab — Step-by-Step Flow

Step	Action
1	View original system prompt (syntax-highlighted read-only)
2	Edit & select recommendations to apply
3	Apply → generate enhanced prompt; view diff or syntax-highlighted result
4 (Evaluate)	Paste both prompts + test inputs, pick metrics, run LLM-judge with per-model config
5 (Suggest)	Generate priority-ranked suggestions grounded in judge score deltas
6 (Arena)	Head-to-head DeepEval arena comparison with natural-language criteria



3.4 Actions Layer — ui/actions/

recommendations.py

- `fetch_recommendations()` — score prompt, populate criteria + recs
- `apply_recommendations()` — generate enhanced prompt
- `fetch_behavioral_recommendations()` — analyze trace failures

evaluation.py

- `run_enhance_evaluation()` — LLM-judge eval
- `generate_enhance_suggestions()` — priority suggestions
- `run_deepeval_prompt_metrics()` — GEval metrics
- `apply_judge_recommendations()` — apply + refine

3.5 Key Components — ui/components.py

- `render_prompt_html()` — syntax-highlighted, scrollable prompt viewer with line numbers
- `render_section_header()` — step-numbered section headers using global CSS classes
- `render_recommendation_checkboxes()` — interactive checkbox list for selecting recommendations
- `render_scores()` — total score card + per-criteria metric widgets
- `render_diff()` — red/green diff view between original and enhanced prompt
- `render_deepeval_results()` — test case results with pass/fail, latency, metric scores
- `_build_diff_html()` — cached pure function for diff computation (via `@st.cache_data`)

3.6 API Client — ui/api_client.py

Retry-enabled HTTP client

All backend calls use exponential back-off retry (max 3 attempts, 1s base wait). Retries on `ConnectionError`, `Timeout`, `HTTP 429`, and `HTTP 5xx`. Shows toast notification on each retry.



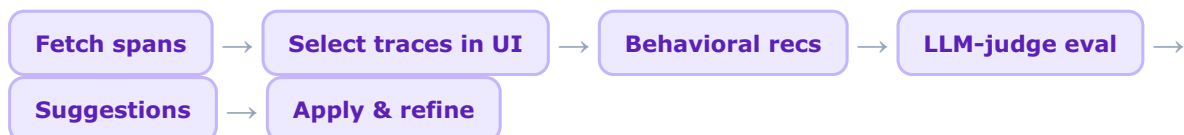
4. Trace Module (trace/)

Fetches and parses live production spans from TrueFoundry for use as evaluation examples.

`trace_parser.py`

- **TraceInput** dataclass — `span_id`, `trace_id`, `timestamp`, `system_prompt`, `user_message`, `trace_output`, `model_name`, `application`, `latency_ms`, `cost_usd`, `prompt_fqn`
- **fetch_live_spans(hours, limit, fqn_filter, tfy_host, tfy_api_key)** — queries TFY API for recent ChatCompletion spans
- **parse_spans_to_inputs(spans)** — converts raw spans to structured TraceInput list; skips spans missing required fields

Trace Evaluation Flow



5. CLI (cli/)

`main.py`

Interactive terminal UI (Rich). Fetches traces, shows selection table, runs the 3-step evaluation pipeline end-to-end.

`pipeline.py`

Three pipeline steps: score original prompt → apply recommendations → LLM-judge comparison. Each step calls `POST /chat`.

6. System Prompts (prompts/)

File	Used For
<code>get_recommendation_system.txt</code>	Score prompt on 5 criteria; generate recommendations
<code>get_recommendation_user.txt</code>	User-turn template for scoring request
<code>apply_recommendation_system.txt</code>	Generate enhanced prompt from recommendations
<code>apply_recommendation_user.txt</code>	User-turn template for enhancement request
<code>behavioral_recommendations_system.txt</code>	Analyze trace examples to identify behavioral gaps
<code>generate_suggestions_system.txt</code>	Generate priority-ranked improvement suggestions from judge results



7. Configuration (config/)

config/application.yaml — Key Sections

```
application_details:
  ml_repo: "new-repo"

llm:
  model_name: "tfy-ai-vertex/gemini-3-flash-preview"

assistants:
  get_recommendation:
    prompt_template_id: "chat_prompt:truefoundry/new-repo/get_recommendation:8"
  apply_recommendation:
    prompt_template_id: "chat_prompt:truefoundry/new-repo/apply_recommendation:8"
  prompt_test_eval_recommendation:
    prompt_template_id: "chat_prompt:truefoundry/new-repo/verify_tests:1"

metrics:
  prompt_metrics:
    correctness_geval:
      threshold: 0.7

checkpointer:
  type: "in_memory" # options: in_memory | redis | postgres

redis:
  host: "localhost"
  port: 6380
```

State Persistence Options

in_memory (default)

MemorySaver — suitable for dev/testing. No external dependency.

redis / postgres

Persistent checkpointing for production. Redis uses async pool manager; Postgres uses async connection.



8. Installation

Prerequisites

Python 3.12+

uv (recommended)

TrueFoundry account

Redis (optional)

PostgreSQL (optional)

Step 1 — Clone & install dependencies

```
# Clone the repo
git clone <repo-url>
cd prompt-validator

# Install with uv (recommended)
uv sync

# OR with pip inside a virtualenv
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Step 2 — Set up environment variables

Create a `.env` file in the project root with the following keys:

```
LLM_BASE_URL=https://<your-tyf-host>/api/llm
TFY_HOST=https://<your-tyf-host>
TFY_API_KEY=<your-truefoundry-api-key>
CONFIGBASEPATH=./config
LLM_MODEL=tfy-ai-vertex/gemini-3-flash-preview
```

Variable	Description	Required
TFY_HOST	Your TrueFoundry tenant URL (e.g. <code>https://company.truefoundry.com</code>)	Yes
TFY_API_KEY	TrueFoundry API key — generate from the TFY console under Settings → API Keys	Yes
LLM_BASE_URL	TrueFoundry LLM gateway base URL for model inference	Yes
LLM_MODEL	Default model FQN used when no override is specified in the UI	Yes
CONFIGBASEPATH	Path to the config/ directory. Defaults to <code>./config</code>	No

Step 3 — Update config/application.yaml

Set your TrueFoundry prompt FQNs and ML repo name under `assistants` and `application_details`.



```
application_details:
  ml_repo: "your-ml-repo-name"

assistants:
  get_recommendation:
    prompt_template_id: "chat_prompt:truefoundry/<your-repo>/get_recommendation:latest"
  apply_recommendation:
    prompt_template_id: "chat_prompt:truefoundry/<your-repo>/apply_recommendation:latest"
```

Installation Tips

- **uv vs pip:** Use `uv sync` for faster, reproducible installs. It respects `pyproject.toml` and auto-creates the `virtualenv`.
- **DeepEval setup:** Run `deepeval login` once to configure your Confident AI key, or skip if not using cloud tracing.
- **Redis / Postgres checkpointing:** Only needed for production multi-user deployments. Set `checkpointer.type` in `application.yaml`. Default is `in_memory` which works fine for local use.
- **TrueFoundry prompts:** Push your prompt files using the TFY CLI before running. The app resolves prompts by FQN at runtime — no redeployment needed for prompt changes.
- **Port conflicts:** Backend defaults to `:21120`. Frontend (Streamlit) defaults to `:8501`. Change via CLI flags if needed.
- **Weasyprint (PDF export):** Requires system libraries on Linux — install via `apt install libpango-1.0-0 libpangoft2-1.0-0` if PDF generation fails.

9. Commands to Run

Start the Backend (FastAPI)

```
uvicorn src.chat.controller.chat_controller:app \
  --host 0.0.0.0 --port 21120 --reload
```

API: <http://localhost:21120>

Swagger: <http://localhost:21120/docs>

ReDoc: <http://localhost:21120/redoc>

Start the Frontend (Streamlit)

```
streamlit run ui/app.py --server.port 8501
```

UI: <http://localhost:8501>

Docker (optional)

```
# Build image
docker build -t prompt-enhancement-accelerator .
```



```
# Run with docker-compose
docker-compose up
```

Command	What it does
<code>uv sync</code>	Install all dependencies from pyproject.toml into .venv
<code>uv add <pkg></code>	Add a new dependency and update pyproject.toml
<code>uvicorn ... --reload</code>	Start backend with hot-reload on file changes (dev only)
<code>streamlit run ui/app.py</code>	Start the Streamlit UI on :8501
<code>deepeval login</code>	Authenticate with Confident AI for DeepEval cloud tracing

10. Key Dependencies

langchain

langgraph

fastapi

uvicorn

streamlit

weasyprint

deepeval

truefoundry-sdk

pydantic

redis

asyncpg

nest_asyncio

pandas

requests